Numpy_crash_course_4th, 5th & 6th-03-2026

04-03-2026

In [1]:
```
import numpy as np
```

In [2]:
```
np.__version__
```

Out[2]:
```
'2.3.5'
```

In [3]:
```
my_list = [0,1,2,3,4,5]
my_list
```

Out[3]:
```
[0, 1, 2, 3, 4, 5]
```

In [4]:
```
type(my_list)
```

Out[4]:
```
list
```

In [5]:
```
arr = np.array(my_list)
arr
```

Out[5]:
```
array([0, 1, 2, 3, 4, 5])
```

In [6]:
```
type(arr)
```

Out[6]:
```
numpy.ndarray
```

ndarray means N-Dimentional array

05-03-2026

In [7]:
```
np.arange(10) # arange -- it will print the value within interval
```

Out[7]:
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [8]:
```
np.arange(10, 15)
```

Out[8]:
```
array([10, 11, 12, 13, 14])
```

In [9]:
```
np.arange(10, 20, 5)
```

Out[9]:
```
array([10, 15])
```

In [10]:
```
np.arange(10, 20, 5, 2) # gives error it functions almost about range
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[10], line 1
----> 1 np.arange(10, 20, 5, 2) # gives error it functions almost about range

TypeError: Cannot interpret '2' as a data type
```

In [11]:
```
np.zeros(5) # one dimentional array with zeros | parameter tunning
```

Out[11]:
```
array([0., 0., 0., 0., 0.])
```

In [12]:
```
np.zeros(5, dtype=int) # hyperparameter tunning
```

Out[12]:
```
array([0, 0, 0, 0, 0])
```

In [13]:
```
np.zeros([2,2], dtype = int) #2d array
```

Out[13]:
```
array([[0, 0],
       [0, 0]])
```

In [14]:
```
np.zeros([5,10], dtype = int)
```

Out[14]:
```
array([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]])
```

In [15]:
```
np.ones(3)
```

Out[15]:
```
array([1., 1., 1.])
```

In [16]:
```
np.ones((3,5), dtype = int)
```

Out[16]:
```
array([[1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1]])
```

In [17]:
```
np.arange(10,20)
```

Out[17]:
```
array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19])
```

In [18]:
```
np.arange(20, 10)
```

Out[18]:
```
array([], dtype=int64)
```

In [19]:
```
np.arange(-20, 10)
```

Out[19]:
```
array([-20, -19, -18, -17, -16, -15, -14, -13, -12, -11, -10,  -9,  -8,
        -7,  -6,  -5,  -4,  -3,  -2,  -1,   0,   1,   2,   3,   4,   5,
         6,   7,   8,   9])
```

In [20]:
```
np.twos(3) # it throws error, only zeros and ones or calles remaining are not the function of
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[20], line 1
----> 1 np.twos(3) # it throws error, only zeros and ones or calles remaining are not the func
tion of numpy

File ~\anaconda3\Lib\site-packages\numpy\__init__.py:795, in __getattr__(attr)
    792     import numpy.char as char
    793     return char.chararray
--> 795 raise AttributeError(f"module {__name__!r} has no attribute {attr!r}")

AttributeError: module 'numpy' has no attribute 'twos'
```

In [21]:
```
random.rand(5)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[21], line 1
----> 1 random.rand(5)

NameError: name 'random' is not defined
```

In [22]:
```
np.random.rand(5)
```

Out[22]:
```
array([0.79738447, 0.11571239, 0.74433108, 0.85731495, 0.13659933])
```

In [23]:
```
np.random.rand(2)
```

Out[23]:
```
array([0.9607006 , 0.24808468])
```

In [24]:
```
np.random.rand(3,2)
```

Out[24]:
```
array([[0.80571786, 0.94084721],
       [0.01872823, 0.11349256],
       [0.51302557, 0.19841625]])
```

In [25]:
```
np.random.rand(3)
```

Out[25]:
```
array([0.75508536, 0.72302605, 0.86598466])
```

In [26]:
```
np.random.randint(3)
```

Out[26]:
```
1
```

In [27]:
```
np.random.randint(5)
```

Out[27]:
```
3
```

In [28]:
```
np.random.randint(5,10,3) #generating 3 random numbers b/w 5 to 10
```

Out[28]:
```
array([7, 6, 8], dtype=int32)
```

In [29]:
```
ran = np.random.randint(5,10,3)
print(ran)
```

```
[5 6 8]
```

In [30]:
```
otp = np.random.randint(5,10,4)
print(otp)
```

```
[9 9 5 5]
```

In [31]:
```
otp = np.random.randint(5,10,4)
print(otp)
```

```
[9 6 7 5]
```

In [32]:
```
otp = np.random.randint(5,10,4)
print(otp)
```

```
[8 8 7 8]
```

In [33]:
```
arr1 = np.random.randint(10,40,(10,10))
print(arr1)
```

```
[[31 10 24 17 31 21 27 11 38 15]
 [33 12 24 37 20 14 31 25 18 10]
 [38 21 28 38 16 27 35 16 23 18]
 [28 24 36 13 35 33 29 25 33 26]
 [33 17 31 12 34 20 28 32 17 18]
 [39 38 19 38 36 22 38 29 20 20]
 [17 17 24 33 37 21 38 17 20 13]
 [16 26 16 27 28 20 38 16 19 17]
 [37 15 28 31 25 36 25 35 25 26]
 [36 32 24 27 21 21 35 31 21 23]]
```

In [34]:
```
arr1 = np.random.randint(10,40,(10,10))
print(arr1)
```

```
[[19 31 10 35 27 30 10 37 36 23]
 [22 16 28 34 20 19 33 16 23 28]
 [39 14 13 19 19 38 34 23 10 23]
 [36 22 15 23 29 24 39 12 39 13]
 [36 12 19 19 19 37 31 10 24 33]
 [13 28 24 23 30 39 38 36 36 26]
 [26 31 38 23 13 24 32 10 29 22]
 [38 24 36 23 28 36 19 34 37 38]
 [23 21 16 21 28 34 37 37 22 31]
 [39 32 14 13 22 35 27 10 34 22]]
```

In [35]:
```
arr1
```

Out[35]:
```
array([[19, 31, 10, 35, 27, 30, 10, 37, 36, 23],
       [22, 16, 28, 34, 20, 19, 33, 16, 23, 28],
       [39, 14, 13, 19, 19, 38, 34, 23, 10, 23],
       [36, 22, 15, 23, 29, 24, 39, 12, 39, 13],
       [36, 12, 19, 19, 19, 37, 31, 10, 24, 33],
       [13, 28, 24, 23, 30, 39, 38, 36, 36, 26],
       [26, 31, 38, 23, 13, 24, 32, 10, 29, 22],
       [38, 24, 36, 23, 28, 36, 19, 34, 37, 38],
       [23, 21, 16, 21, 28, 34, 37, 37, 22, 31],
       [39, 32, 14, 13, 22, 35, 27, 10, 34, 22]], dtype=int32)
```

In [36]:
```
arr1 = np.random.randint(1,5,(3,3))
print(arr1)
```

```
[[4 1 1]
 [4 4 1]
 [4 4 2]]
```

indexing, slicing in array

In [37]:
```
arr1[:]
```

Out[37]:
```
array([[4, 1, 1],
       [4, 4, 1],
       [4, 4, 2]], dtype=int32)
```

In [38]:
```
len(arr1)
```

Out[38]:
```
3
```

In [39]:
```
arr1.shape
```

Out[39]:
```
(3, 3)
```

In [40]:
```
arr0 = np.random.randint(10,40,(10,10))
print(arr0)
```

```
[[10 20 29 12 17 13 35 28 11 30]
 [15 38 26 14 32 15 33 29 18 12]
 [36 13 22 28 36 33 10 31 30 28]
 [11 22 32 37 37 24 37 22 12 20]
 [29 19 10 24 33 21 13 12 23 30]
 [18 36 13 13 19 21 12 20 33 31]
 [12 16 11 25 14 31 18 29 18 13]
 [15 29 18 17 32 35 20 17 24 32]
 [32 12 31 22 35 13 33 22 32 36]
 [32 12 36 31 14 14 37 13 16 17]]
```

In [41]:
```
arr0.shape
```

Out[41]:
```
(10, 10)
```

In [42]:
```
len(arr0)
```

Out[42]:
```
10
```

In [43]:
```
arr1
```

Out[43]:
```
array([[4, 1, 1],
       [4, 4, 1],
       [4, 4, 2]], dtype=int32)
```

In [44]:
```
arr1[0] # prints 0th row
```

Out[44]:
```
array([4, 1, 1], dtype=int32)
```

In [45]:
```
arr1[1]
```

Out[45]:
```
array([4, 4, 1], dtype=int32)
```

In [46]:
```
arr1[4] #index error because we have only 3 rows
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[46], line 1
----> 1 arr1[4] #index error because we have only 3 rows

IndexError: index 4 is out of bounds for axis 0 with size 3
```

In [47]:
```
arr1[:,0] #it print column
```

Out[47]:
```
array([4, 4, 4], dtype=int32)
```

In [48]:
```
arr1[:,1]
```

Out[48]:
```
array([1, 4, 4], dtype=int32)
```

In [49]:
```
arr1[:,2]
```

Out[49]:
```
array([1, 1, 2], dtype=int32)
```

In [50]:
```
arr1
```

Out[50]:
```
array([[4, 1, 1],
       [4, 4, 1],
       [4, 4, 2]], dtype=int32)
```

In [51]:
```
arr1[0,0]
```

Out[51]:
```
np.int32(4)
```

In [52]:
```
arr1[1,2]
```

Out[52]:
```
np.int32(1)
```

In [53]:
```
v = arr1[1, 2]
print(v)
```

```
1
```

In [54]:
```
arr0
```

Out[54]:
```
array([[10, 20, 29, 12, 17, 13, 35, 28, 11, 30],
       [15, 38, 26, 14, 32, 15, 33, 29, 18, 12],
       [36, 13, 22, 28, 36, 33, 10, 31, 30, 28],
       [11, 22, 32, 37, 37, 24, 37, 22, 12, 20],
       [29, 19, 10, 24, 33, 21, 13, 12, 23, 30],
       [18, 36, 13, 13, 19, 21, 12, 20, 33, 31],
       [12, 16, 11, 25, 14, 31, 18, 29, 18, 13],
       [15, 29, 18, 17, 32, 35, 20, 17, 24, 32],
       [32, 12, 31, 22, 35, 13, 33, 22, 32, 36],
       [32, 12, 36, 31, 14, 14, 37, 13, 16, 17]], dtype=int32)
```

In [55]:
```
arr0[-7]
```

Out[55]:
```
array([11, 22, 32, 37, 37, 24, 37, 22, 12, 20], dtype=int32)
```

In [56]:
```
arr0[:,-7]
```

Out[56]:
```
array([12, 14, 28, 37, 24, 13, 25, 17, 22, 31], dtype=int32)
```

In [57]:
```
arr0[3:4, 2:6]
# 3:4 belongs to row & 2:6 belongs to column then 3:4 means from 3 to (n-1)-->(4-1) = 3 then
```

Out[57]:
```
array([[32, 37, 37, 24]], dtype=int32)
```

In [58]:
```
arr1
```

Out[58]:
```
array([[4, 1, 1],
       [4, 4, 1],
       [4, 4, 2]], dtype=int32)
```

In [59]:
```
arr1[1]
```

Out[59]:
```
array([4, 4, 1], dtype=int32)
```

In [60]:
```
arr1[1,2] # in this 1 means first row & 2 means cloumn
```

Out[60]:
```
np.int32(1)
```

In [61]:
```
arr1[1:2] # in this 1 means 1st index in row and 2 means (2-1)--> 1st index in row
```

Out[61]:
```
array([[4, 4, 1]], dtype=int32)
```

06-03-2026

In [62]:
```
print(arr)
```

```
[0 1 2 3 4 5]
```

In [63]:
```
print(arr1)
```

```
[[4 1 1]
 [4 4 1]
 [4 4 2]]
```

In [64]:
```
print(arr0)
```

```
[[10 20 29 12 17 13 35 28 11 30]
 [15 38 26 14 32 15 33 29 18 12]
 [36 13 22 28 36 33 10 31 30 28]
 [11 22 32 37 37 24 37 22 12 20]
 [29 19 10 24 33 21 13 12 23 30]
 [18 36 13 13 19 21 12 20 33 31]
 [12 16 11 25 14 31 18 29 18 13]
 [15 29 18 17 32 35 20 17 24 32]
 [32 12 31 22 35 13 33 22 32 36]
 [32 12 36 31 14 14 37 13 16 17]]
```

In [66]:
```
arr0[::1]
```

Out[66]:
```
array([[10, 20, 29, 12, 17, 13, 35, 28, 11, 30],
       [15, 38, 26, 14, 32, 15, 33, 29, 18, 12],
       [36, 13, 22, 28, 36, 33, 10, 31, 30, 28],
       [11, 22, 32, 37, 37, 24, 37, 22, 12, 20],
       [29, 19, 10, 24, 33, 21, 13, 12, 23, 30],
       [18, 36, 13, 13, 19, 21, 12, 20, 33, 31],
       [12, 16, 11, 25, 14, 31, 18, 29, 18, 13],
       [15, 29, 18, 17, 32, 35, 20, 17, 24, 32],
       [32, 12, 31, 22, 35, 13, 33, 22, 32, 36],
       [32, 12, 36, 31, 14, 14, 37, 13, 16, 17]], dtype=int32)
```

In [67]:
```
arr0[::2]
```

Out[67]:
```
array([[10, 20, 29, 12, 17, 13, 35, 28, 11, 30],
       [36, 13, 22, 28, 36, 33, 10, 31, 30, 28],
       [29, 19, 10, 24, 33, 21, 13, 12, 23, 30],
       [12, 16, 11, 25, 14, 31, 18, 29, 18, 13],
       [32, 12, 31, 22, 35, 13, 33, 22, 32, 36]], dtype=int32)
```

In [68]:
```
arr0[::3]
```

Out[68]:
```
array([[10, 20, 29, 12, 17, 13, 35, 28, 11, 30],
       [11, 22, 32, 37, 37, 24, 37, 22, 12, 20],
       [12, 16, 11, 25, 14, 31, 18, 29, 18, 13],
       [32, 12, 36, 31, 14, 14, 37, 13, 16, 17]], dtype=int32)
```

In [69]:
```
arr0
```

Out[69]:
```
array([[10, 20, 29, 12, 17, 13, 35, 28, 11, 30],
       [15, 38, 26, 14, 32, 15, 33, 29, 18, 12],
       [36, 13, 22, 28, 36, 33, 10, 31, 30, 28],
       [11, 22, 32, 37, 37, 24, 37, 22, 12, 20],
       [29, 19, 10, 24, 33, 21, 13, 12, 23, 30],
       [18, 36, 13, 13, 19, 21, 12, 20, 33, 31],
       [12, 16, 11, 25, 14, 31, 18, 29, 18, 13],
       [15, 29, 18, 17, 32, 35, 20, 17, 24, 32],
       [32, 12, 31, 22, 35, 13, 33, 22, 32, 36],
       [32, 12, 36, 31, 14, 14, 37, 13, 16, 17]], dtype=int32)
```

In [70]:
```
arr0[0:10:5]
```

Out[70]:
```
array([[10, 20, 29, 12, 17, 13, 35, 28, 11, 30],
       [18, 36, 13, 13, 19, 21, 12, 20, 33, 31]], dtype=int32)
```

advancing slicing in array

In [71]:
```
arr0[::-2]
```

Out[71]:
```
array([[32, 12, 36, 31, 14, 14, 37, 13, 16, 17],
       [15, 29, 18, 17, 32, 35, 20, 17, 24, 32],
       [18, 36, 13, 13, 19, 21, 12, 20, 33, 31],
       [11, 22, 32, 37, 37, 24, 37, 22, 12, 20],
       [15, 38, 26, 14, 32, 15, 33, 29, 18, 12]], dtype=int32)
```

In [72]:
```
arr0[::-3]
```

Out[72]:
```
array([[32, 12, 36, 31, 14, 14, 37, 13, 16, 17],
       [12, 16, 11, 25, 14, 31, 18, 29, 18, 13],
       [11, 22, 32, 37, 37, 24, 37, 22, 12, 20],
       [10, 20, 29, 12, 17, 13, 35, 28, 11, 30]], dtype=int32)
```

advance slicing done

In [73]:
```
arr1
```

Out[73]:
```
array([[4, 1, 1],
       [4, 4, 1],
       [4, 4, 2]], dtype=int32)
```

In [74]:
```
arr1[1,2] #when we apply comma it print specific value
```

Out[74]:
```
np.int32(1)
```

In [75]:
```
arr1[1:2] #when we apply collon iti print n-1 formula entire row
```

Out[75]:
```
array([[4, 4, 1]], dtype=int32)
```

reshaping

In [76]:
```
arr
```

Out[76]:
```
array([0, 1, 2, 3, 4, 5])
```

In [77]:
```
arr.reshape(3,2) #it reshape will reshape the matrix into 3/2
```

Out[77]:
```
array([[0, 1],
       [2, 3],
       [4, 5]])
```

In [78]:
```
arr.reshape(4,4) #we can't arrange the size 6 into shape (4,4)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[78], line 1
----> 1 arr.reshape(4,4) #we can't arrange the size 6 into shape (4,4)

ValueError: cannot reshape array of size 6 into shape (4,4)
```

In [79]:
```
arr
```

Out[79]:
```
array([0, 1, 2, 3, 4, 5])
```

In [80]:
```
arr.reshape(3,2) #reshape to 3/2 matrix
```

Out[80]:
```
array([[0, 1],
       [2, 3],
       [4, 5]])
```

In [81]:
```
arr.reshape(2,3) #reshape to 2/3 matrix
```

Out[81]:
```
array([[0, 1, 2],
       [3, 4, 5]])
```

In [82]:
```
arr.reshape(6,1)
```

Out[82]:
```
array([[0],
       [1],
       [2],
       [3],
       [4],
       [5]])
```

In [83]:
```
arr.reshape(1,6)
```

Out[83]:
```
array([[0, 1, 2, 3, 4, 5]])
```

In [84]:
```
arr1
```

Out[84]:
```
array([[4, 1, 1],
       [4, 4, 1],
       [4, 4, 2]], dtype=int32)
```

In [85]:
```
arr1[0:2, 1:3] #0:1 is row and 1:3 is column --> in slicing 2 and 3 will deduce 1 because it
```

Out[85]:
```
array([[1, 1],
       [4, 1]], dtype=int32)
```

Masking or filter in matrix

In [86]:
```
arr0
```

Out[86]:
```
array([[10, 20, 29, 12, 17, 13, 35, 28, 11, 30],
       [15, 38, 26, 14, 32, 15, 33, 29, 18, 12],
       [36, 13, 22, 28, 36, 33, 10, 31, 30, 28],
       [11, 22, 32, 37, 37, 24, 37, 22, 12, 20],
       [29, 19, 10, 24, 33, 21, 13, 12, 23, 30],
       [18, 36, 13, 13, 19, 21, 12, 20, 33, 31],
       [12, 16, 11, 25, 14, 31, 18, 29, 18, 13],
       [15, 29, 18, 17, 32, 35, 20, 17, 24, 32],
       [32, 12, 31, 22, 35, 13, 33, 22, 32, 36],
       [32, 12, 36, 31, 14, 14, 37, 13, 16, 17]], dtype=int32)
```

In [87]:
```
arr0 < 20
```

Out[87]:
```
array([[ True, False, False,  True,  True,  True, False, False,  True,
        False],
       [ True, False, False,  True, False,  True, False, False,  True,
         True],
       [False,  True, False, False, False, False,  True, False, False,
        False],
       [ True, False, False, False, False, False, False, False,  True,
        False],
       [False,  True,  True, False, False, False,  True,  True, False,
        False],
       [ True, False,  True,  True,  True, False,  True, False, False,
        False],
       [ True,  True,  True, False,  True, False,  True, False,  True,
         True],
       [ True, False,  True,  True, False, False, False,  True, False,
        False],
       [False,  True, False, False, False,  True, False, False, False,
        False],
       [False,  True, False, False,  True,  True, False,  True,  True,
         True]])
```

In [88]:
```
arr0[arr0 < 20]
```

Out[88]:
```
array([10, 12, 17, 13, 11, 15, 14, 15, 18, 12, 13, 10, 11, 12, 19, 10, 13,
       12, 18, 13, 13, 19, 12, 12, 16, 11, 14, 18, 18, 13, 15, 18, 17, 17,
       12, 13, 12, 14, 14, 13, 16, 17], dtype=int32)
```

In [89]:
```
arr0[arr0 == 20]
```

Out[89]:
```
array([20, 20, 20, 20], dtype=int32)
```

In [90]:
```
arr0[arr0 != 20]
```

Out[90]:
```
array([10, 29, 12, 17, 13, 35, 28, 11, 30, 15, 38, 26, 14, 32, 15, 33, 29,
       18, 12, 36, 13, 22, 28, 36, 33, 10, 31, 30, 28, 11, 22, 32, 37, 37,
       24, 37, 22, 12, 29, 19, 10, 24, 33, 21, 13, 12, 23, 30, 18, 36, 13,
       13, 19, 21, 12, 33, 31, 12, 16, 11, 25, 14, 31, 18, 29, 18, 13, 15,
       29, 18, 17, 32, 35, 17, 24, 32, 32, 12, 31, 22, 35, 13, 33, 22, 32,
       36, 32, 12, 36, 31, 14, 14, 37, 13, 16, 17], dtype=int32)
```

In [91]:
```
arr0[arr0 > 40] # we have no values greather than 40
```

Out[91]:
```
array([], dtype=int32)
```

In [92]:
```
arr0[arr0 > 35]
```

Out[92]:
```
array([38, 36, 36, 37, 37, 37, 36, 36, 36, 37], dtype=int32)
```

In [93]:
```
arr
```

Out[93]: array([0, 1, 2, 3, 4, 5])

In [94]:
```
max(arr)
```

Out[94]: np.int64(5)

In [95]:
```
max(arr)
```

Out[95]: np.int64(5)

POC : proof of concept (image to array)

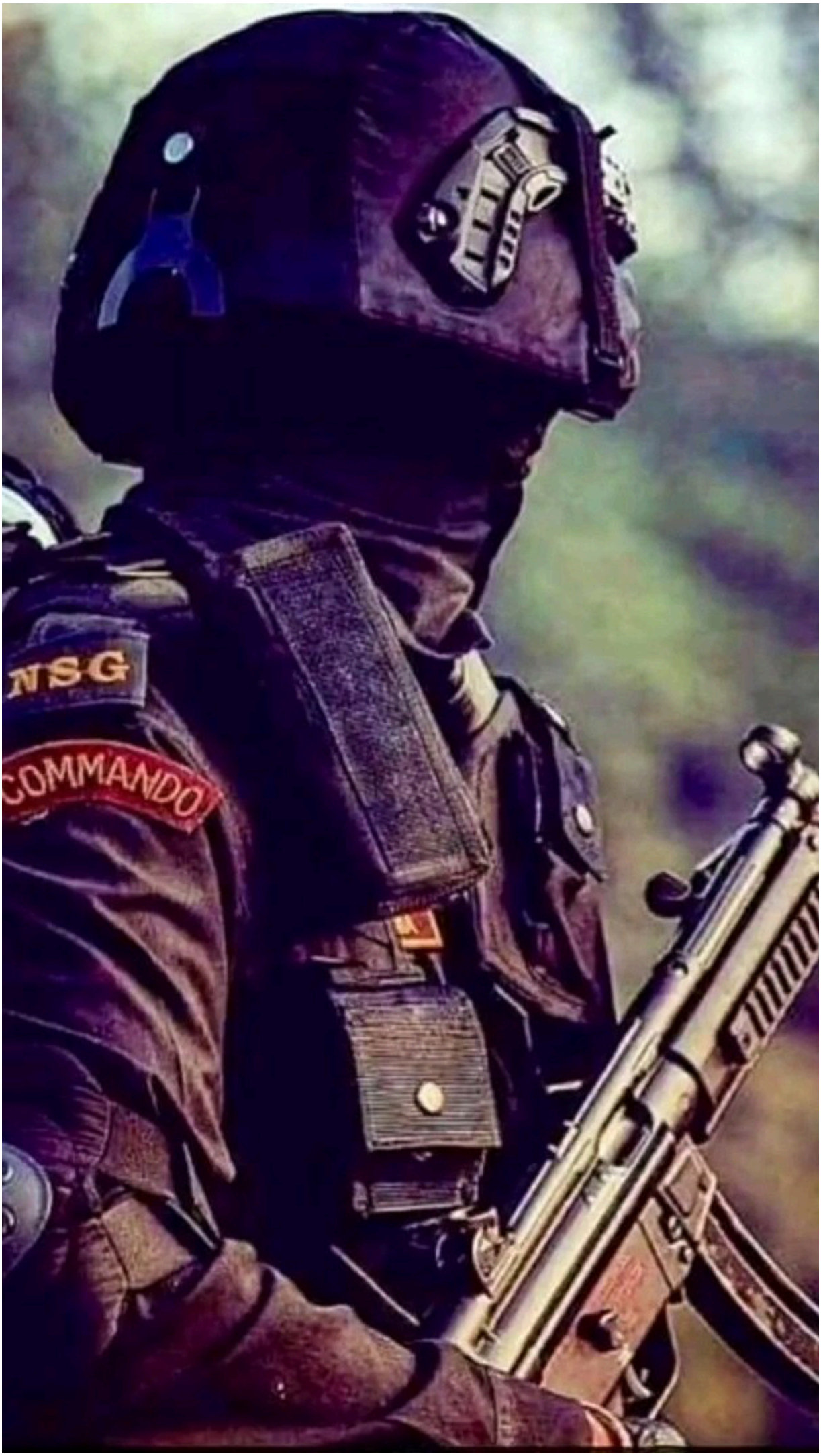
In [96]:
```
import matplotlib.pyplot as plt # use for visualization
```

In [97]:
```
from PIL import Image #PIL means python imaging library -- I can upload image by PIL
```

In [98]:
```
NSG = Image.open(r'C:\Users\mehar\OneDrive\Desktop\NSG.jpg')
```

In [99]:
```
NSG
```

Out[99]:



In [100…

```
type(NSG)
```

Out[100…

```
PIL.JpegImagePlugin.JpegImageFile
```

In [101…

```
NSG_arr = np.asarray(NSG)
NSG_arr
```

Out[101…

```
array([[[209, 204, 175],
        [207, 202, 173],
        [205, 201, 174],
        ...,
        [133, 148, 153],
        [131, 146, 151],
        [129, 144, 149]],

       [[209, 204, 175],
        [207, 202, 173],
        [205, 199, 173],
        ...,
        [133, 148, 153],
        [130, 145, 150],
        [128, 143, 148]],

       [[208, 203, 174],
        [207, 202, 173],
        [205, 200, 171],
        ...,
        [133, 148, 153],
        [129, 144, 149],
        [126, 141, 146]],

       ...,

       [[  2,   1,   9],
        [  2,   1,   9],
        [  2,   1,   9],
        ...,
        [ 13,   0,  12],
        [ 13,   0,  12],
        [ 13,   0,  12]],

       [[  1,   0,   6],
        [  1,   0,   6],
        [  1,   0,   6],
        ...,
        [ 12,   0,  10],
        [ 12,   0,  10],
        [ 12,   0,  10]],

       [[  0,   0,   5],
        [  0,   0,   5],
        [  0,   0,   4],
        ...,
        [ 12,   1,   9],
        [ 12,   1,   9],
        [ 12,   1,   9]]], shape=(1080, 608, 3), dtype=uint8)
```

In [102…

```
plt.imshow(NSG_arr)
```

Out[102…

```
<matplotlib.image.AxesImage at 0x24d814e56a0>
```

In [103…
```
NSG_arr.shape #if pixel is less then array is less
```

Out[103…
```
(1080, 608, 3)
```

In [ ]:

In [ ]:

In [ ]:

In [ ]: